Project Portfolio

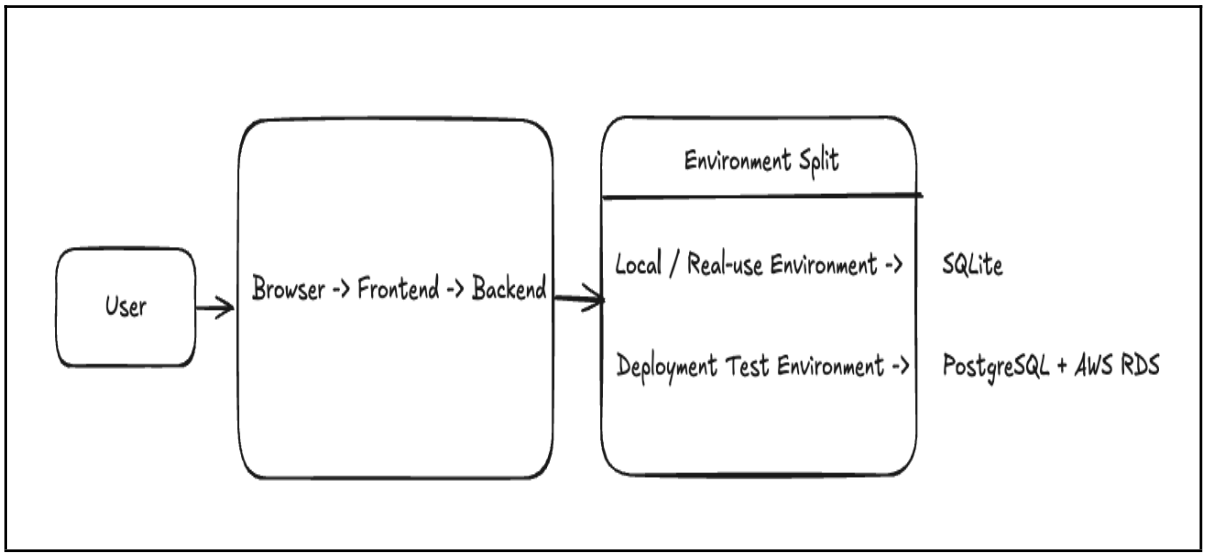
부동산 관리 프로그램

깃허브 : https://github.com/beermoon/Property_manager_prod
주소 : http://3.35.120.200:8080

---

1. 프로젝트 규모에 맞춘 개발·배포 환경 설계

전체적인 아키텍처



문제원인

이 프로그램은 소규모 사용자(1~2명)를 대상으로 하는 프로젝트였기 때문에, 초기부터 AWS 기반으로 상시 운영하기에는 관리와 보안 설정 부담이 크다고 판단했습니다.

프로젝트 규모에 비해 인프라 구성이 과해질 수 있었고, 실제 사용 환경에 더 적합한 방식이 필요했습니다.

따라서 사용자 수와 운영 목적에 맞는 현실적인 환경 구성이 필요했습니다.

해결

실제 사용 버전은 로컬에서 간단하게 실행하고 관리할 수 있도록 SQLite 기반으로 구성했습니다.

동시에 배포 환경도 경험해보기 위해 PostgreSQL과 AWS RDS를 연결한 별도 환경을 테스트했습니다.

이를 통해 실사용 환경과 배포 테스트 환경을 분리하는 방식으로 운영 구조를 설계했습니다.

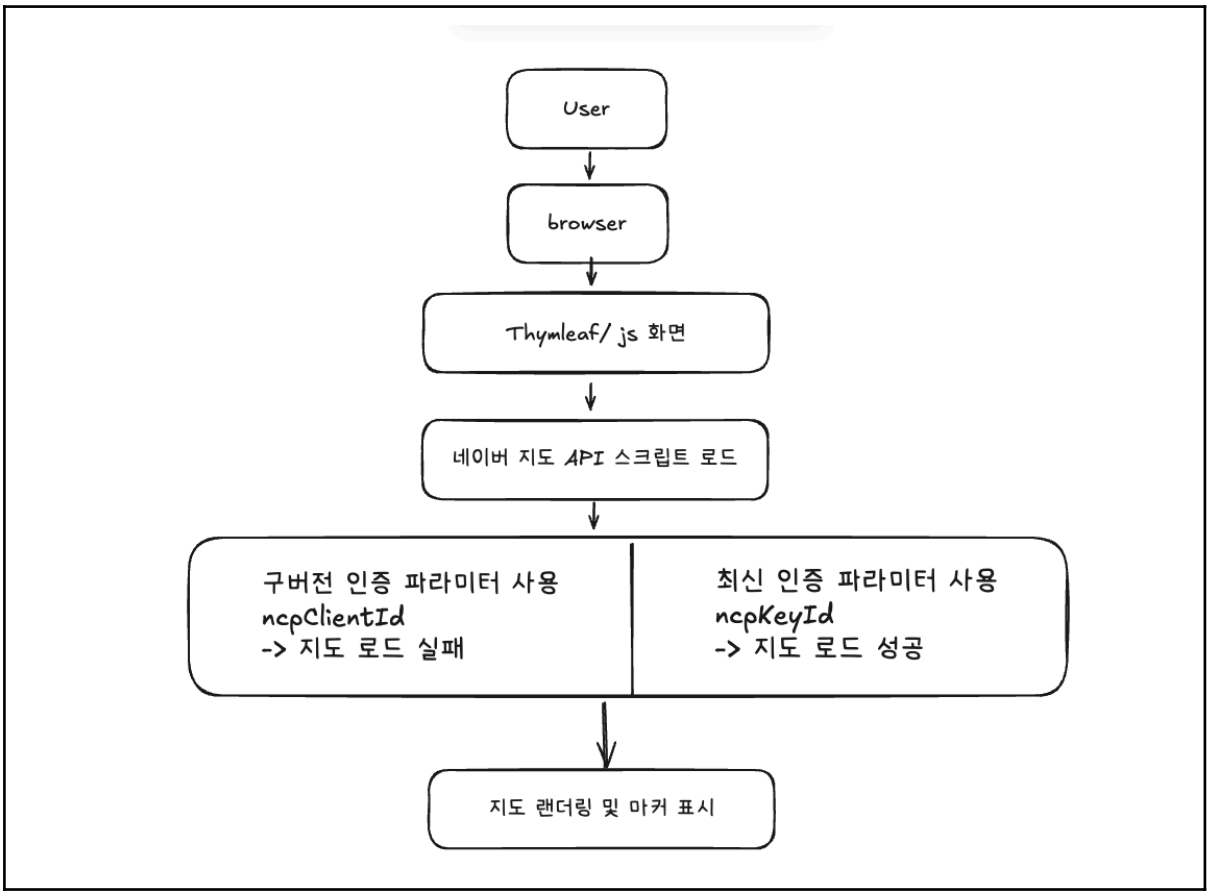
결과

프로젝트 규모에 맞는 운영 방식을 선택해 관리 복잡도를 줄일 수 있었습니다.

실사용 환경은 단순하고 가볍게 유지하면서도, 배포 환경에서 사용하는 데이터베이스 구성도 함께 경험할 수 있었습니다.

결과적으로 현실적인 운영 방식과 확장 가능성을 함께 고려한 구조를 만들 수 있었습니다.

2. 네이버 지도 **API** 연동 오류 해결



문제원인

지도 기반 매물 조회 기능을 구현하는 과정에서 네이버 지도 API가 정상적으로 동작하지 않아, 지도 화면이 표시되지 않는 문제가 발생했습니다.

문제를 분석하는 과정에서 AI를 활용해 오류 가능성을 빠르게 정리하고 원인 후보를 좁혔고, 이후 공식 문서를 다시 확인했습니다.

그 결과 기존 예제와 참고 자료를 바탕으로 사용한 구버전 인증 파라미터 ncpClientId가 현재 연동 방식과 맞지 않아 오류가 발생한 것임을 확인했습니다.

해결 과정

먼저 네이버 지도 API 공식 문서를 기준으로 현재 인증 방식과 스크립트 파라미터 구조를 다시 점검했습니다.

이후 기존 코드에서 사용하던 ncpClientId를 최신 방식인 ncpKeyId로 수정하고, 지도 스크립트가 올바르게 로드되도록 연동 코드를 변경했습니다.

단순히 예제 코드를 그대로 사용하는 것이 아니라, 공식 문서와 실제 코드의 차이를 비교하며 수정하는 방식으로 문제를 해결했습니다.

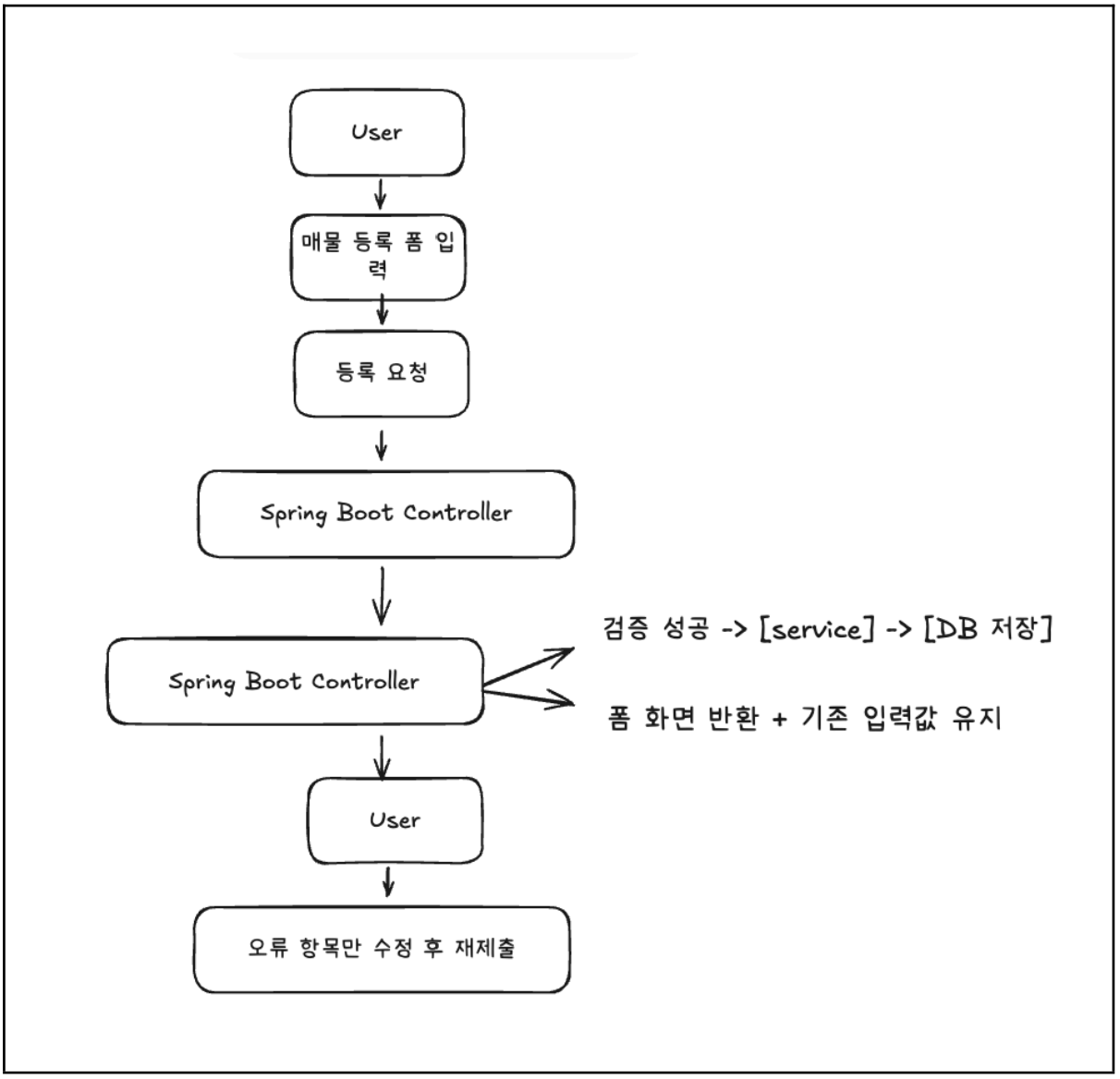
결과

수정 이후 지도 기능이 정상적으로 동작하면서, 매물 위치를 지도 위에서 확인할 수 있는 핵심 기능을 복구할 수 있었습니다.

사용자는 다시 지도 기반으로 매물을 직관적으로 조회할 수 있게 되었고, 프로젝트의 핵심 목적도 유지할 수 있었습니다.

또한 외부 API 연동 문제를 해결할 때는 AI를 활용해 원인 후보를 정리하되, 최종 판단은 공식 문서를 통해 검증하는 과정이 중요하다는 점을 배웠습니다.

3. 등록 폼 검증 실패 시 입력값 유지 처리로 사용자 경험 개선



문제원인

매물 등록 폼에서 검증 오류가 발생하면 사용자가 입력한 값이 모두 초기화되어, 처음부터 다시 작성해야 하는 불편이 있었습니다.

특히 입력 항목이 많은 등록 폼 특성상 일부 값만 잘못 입력해도 전체 내용을 다시 입력해야 했기 때문에 사용자 경험이 좋지 않았습니다.

원인은 검증 실패 후 폼 화면을 다시 반환하는 과정에서 기존 입력값을 다시 화면에 전달하는 처리가 되어 있지 않았기 때문입니다.

해결 과정

먼저 등록 요청 이후 Validation 검사에서 실패하는 흐름을 다시 점검했습니다.

이후 검증 실패 시 단순히 폼 화면만 다시 반환하는 것이 아니라, 사용자가 기존에 입력한 데이터를 함께 전달해 화면에 다시 표시되도록 처리했습니다.

이를 통해 사용자는 전체 값을 다시 입력하지 않고, 오류가 발생한 항목만 수정한 뒤 다시 제출할 수 있도록 개선했습니다.

결과

검증 실패 이후에도 사용자가 입력한 값이 유지되도록 하여 등록 폼의 사용성을 높일 수 있었습니다.

특히 입력 항목이 많은 화면에서 재입력 부담을 줄여 사용자 불편을 완화할 수 있었습니다. 이

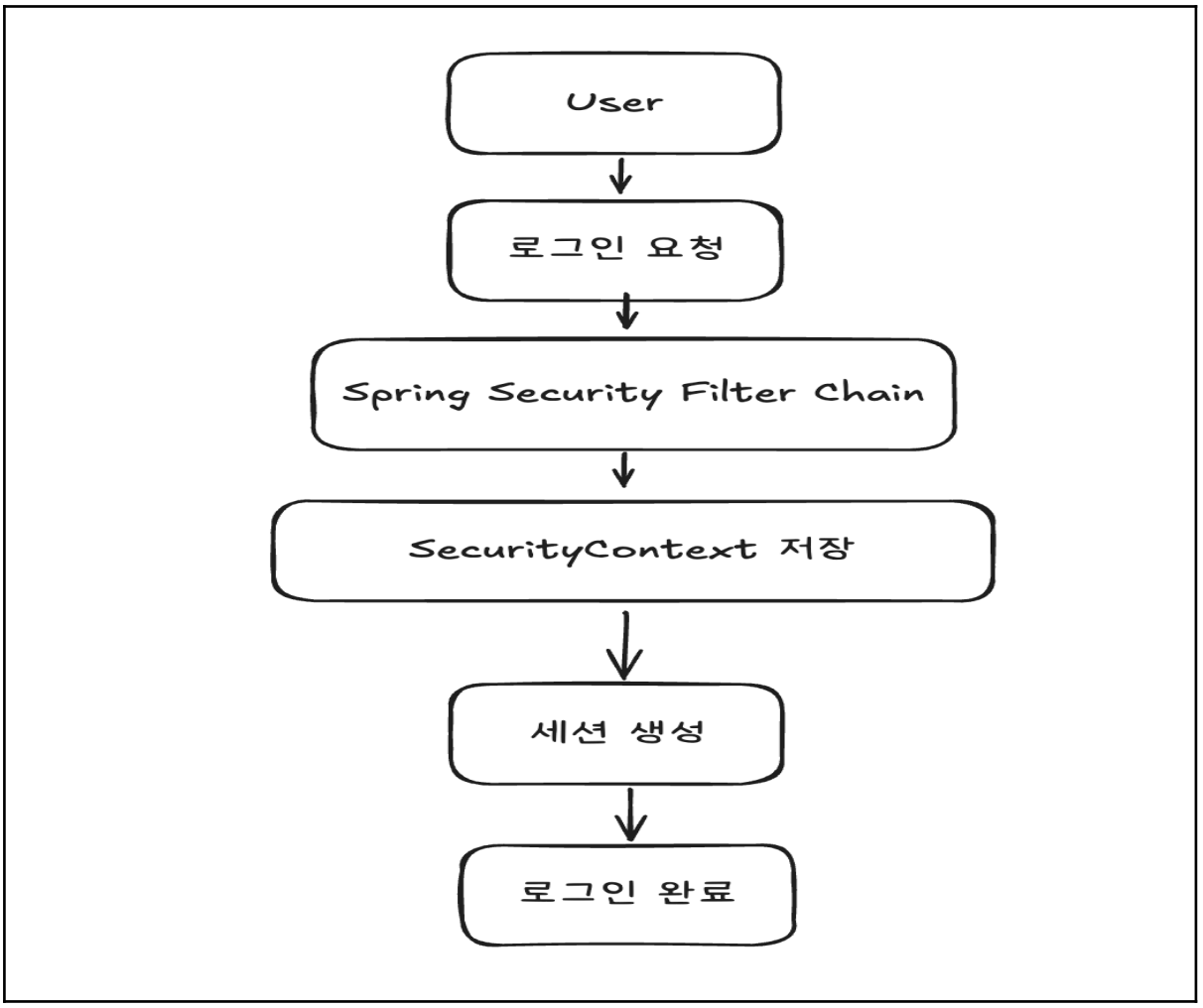
과정을 통해 기능이 정상 동작하는 것뿐 아니라, 오류 상황에서도 사용자가 얼마나 편하게 다시 시도할 수 있는지를 함께 고려하는 것이 중요하다는 점을 배웠습니다.

롯데온 쇼핑몰 프로젝트

깃허브 : https://github.com/greenlotte6/lotte1-lotteon-project-team2 (시연영상 포함)

---

1. 프로젝트 규모에 맞춘 개발·배포 환경 설계



문제원인

초기 로그인 기능은 컨트롤러에서 직접 아이디와 비밀번호를 받아 사용자를 조회하고, 세션에 사용자 정보를 저장하는 방식으로 구현하고 있었습니다.

하지만 Spring Security를 적용한 이후에는 로그인 요청이 컨트롤러까지 도달하기 전에 Security Filter Chain에서 먼저 처리되기 때문에, 기존 방식으로 작성한 로그인 로직이 더 이상 정상적으로 동작하지 않았습니다.

즉, 직접 구현한 로그인 흐름과 Spring Security가 처리하는 인증 흐름이 서로 다른 구조를 가지고 있어 충돌이 발생한 것이 문제였습니다.

해결 과정

이 문제를 해결하기 위해 기존의 컨트롤러 기반 로그인 처리 방식을 제거하고, Spring Security가 제공하는 인증 구조에 맞춰 로그인 흐름을 다시 정리했습니다.

사용자의 로그인 요청이 Filter Chain을 거쳐 Authentication으로 처리되고, 인증이 성공하면 SecurityContext에 사용자 인증 정보가 저장되며, 이후 세션이 생성되는 구조로 전환했습니다.

이를 통해 로그인 기능을 단순히 “직접 구현”하는 방식이 아니라, 보안 프레임워크가 의도한 흐름 안에서 동작하도록 재구성했습니다

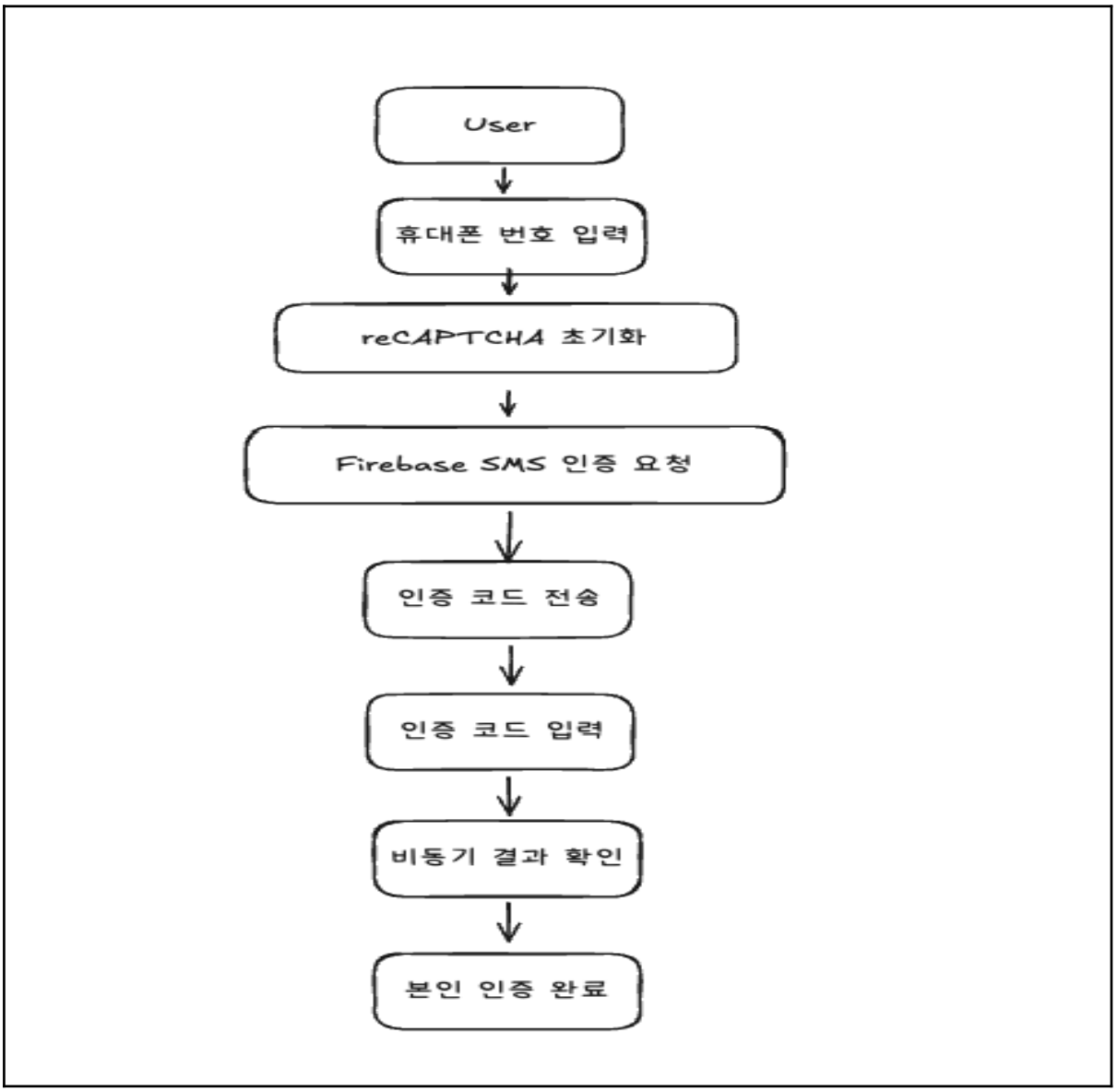
결과

구조 전환 이후 로그인 기능이 Spring Security 기반으로 안정적으로 동작하도록 구현할 수 있었습니다.

또한 인증, 세션, SecurityContext가 어떤 흐름으로 연결되는지 이해하게 되었고, 단순 기능 구현을 넘어 인증 구조 자체를 학습할 수 있었습니다.

이 경험을 통해 보안 기능은 직접 로직을 작성하는 것보다도, 프레임워크의 동작 방식과 책임 분리를 이해하고 그 흐름에 맞게 구현하는 것이 중요하다는 점을 배웠습니다.

2.Firebase 휴대폰 인증 연동 문제 해결



문제원인

회원가입 과정에서 휴대폰 인증 기능을 구현할 때 인증 요청이 정상적으로 동작하지 않는 문제가 발생했습니다.

확인해보니 Firebase 휴대폰 인증은 reCAPTCHA 초기화가 먼저 이루어져야 하는데, 초기화 순서가 맞지 않아 인증 요청이 실패하고 있었습니다.

또한 전화번호를 국제 형식으로 변환하지 않았고, 인증 결과가 비동기적으로 반환되는 구조를 제대로 관리하지 못해 다음 단계에서 오류가 발생하고 있었습니다.

해결 과정

먼저 reCAPTCHA가 인증 요청 전에 정상적으로 초기화되도록 코드 실행 순서를 정리했습니다.

그다음 사용자가 입력한 휴대폰 번호를 Firebase가 요구하는 국제 전화번호 형식인 +82 형태로 변환하도록 처리했습니다.

마지막으로 인증 결과를 전역 변수로 관리해 다음 단계에서도 접근할 수 있도록 수정하면서, 인증 요청부터 코드 확인까지의 흐름이 끊기지 않도록 개선했습니다.

결과

수정 이후 SMS 인증 요청과 인증 코드 확인 과정이 정상적으로 동작하도록 개선할 수 있었습니다.

회원가입 과정에서 휴대폰 본인 인증 기능을 안정적으로 제공할 수 있게 되었고, 외부 인증 서비스를 실제 서비스 흐름에 연결하는 경험을 쌓을 수 있었습니다.

이 과정을 통해 외부 서비스 연동에서는 단순 API 호출뿐 아니라 실행 순서, 데이터 형식, 비동기 흐름까지 함께 고려해야 한다는 점을 배웠습니다.